SYNCHRONISATION EXAMPLES (AND CONDITIONAL VARIABLES)

DINING PHILOSOPHERS PROBLEM

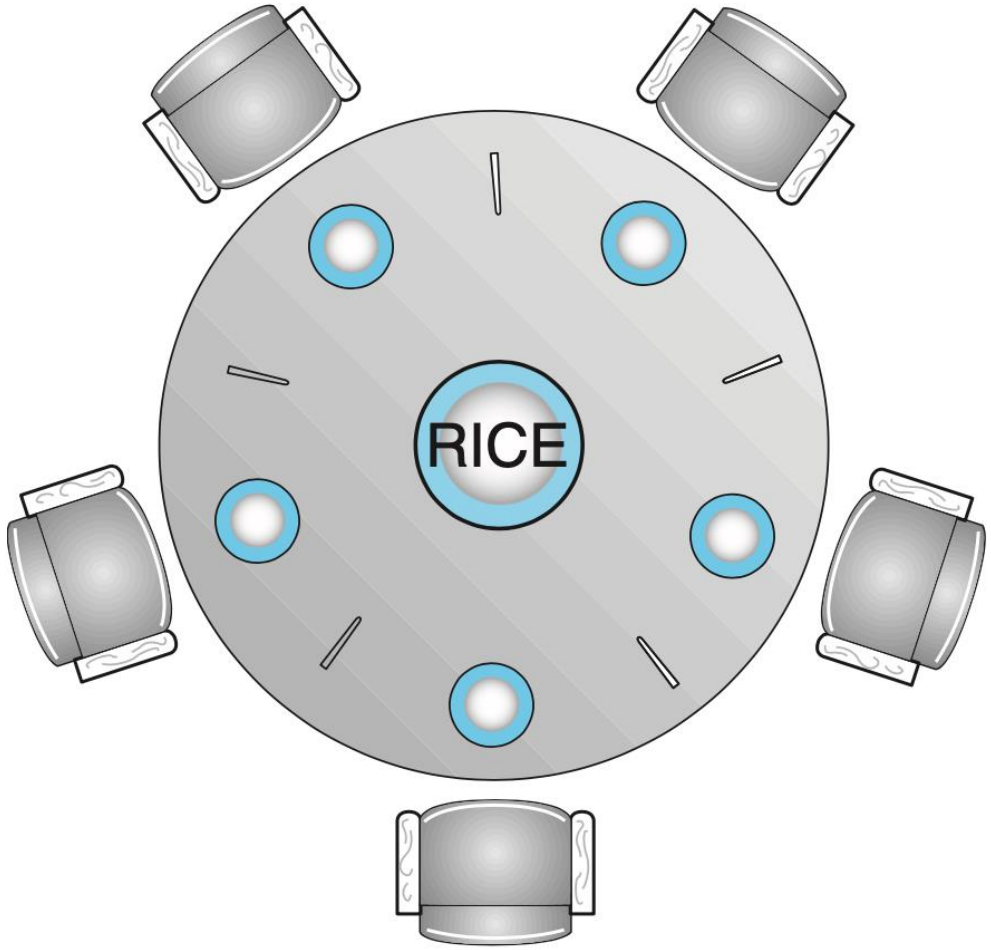
The philosophers spend their time thinking and eating.

A hungry philosopher will try to pick two chop sticks, next to him/her, one at a time.

While eating, the philpsopher will keep them and only put them down when done.

If a chop stick is taken, the philosopher will have to wait until it becomes available.

Let's use one *Thread* per philosopher and one *Lock* per chopstick



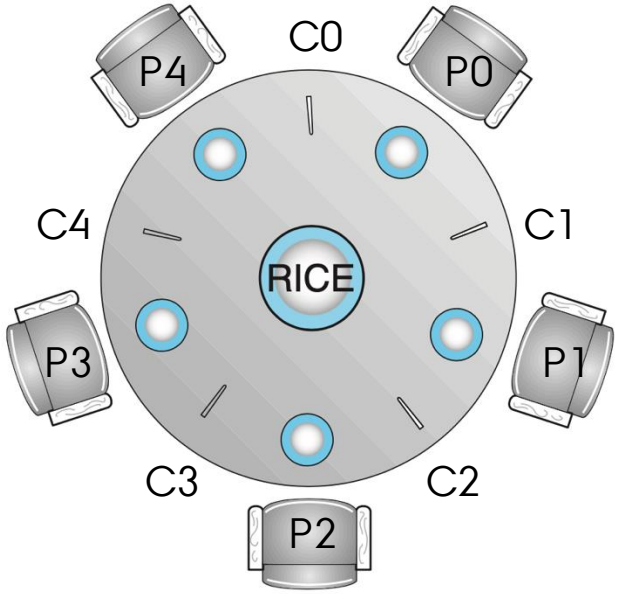
NAIIVE IMPLEMENTATION

```
auto philosopher = [&](int phil)
{
  int right = phil;             // Right chopstick index
  int left = (phil + 1) % 5;    // Left chopstick index
  for (int i = 0; i < 100; i++) // Eat 100 times
  {
    chopstick[right].lock();
    chopstick[left].lock();
    std::cout << "Philosopher " << phil << " is eating" << std::endl;
    std::this_thread::sleep_for(std::chrono::microseconds(1));
    chopstick[left].unlock();
    chopstick[right].unlock();
    std::cout << "Philosopher " << phil << " is sleeping" << std::endl;
  }
};
```

TAKING TURNS

In turn, each Philosopher picks a right fork. Then they pick a left...

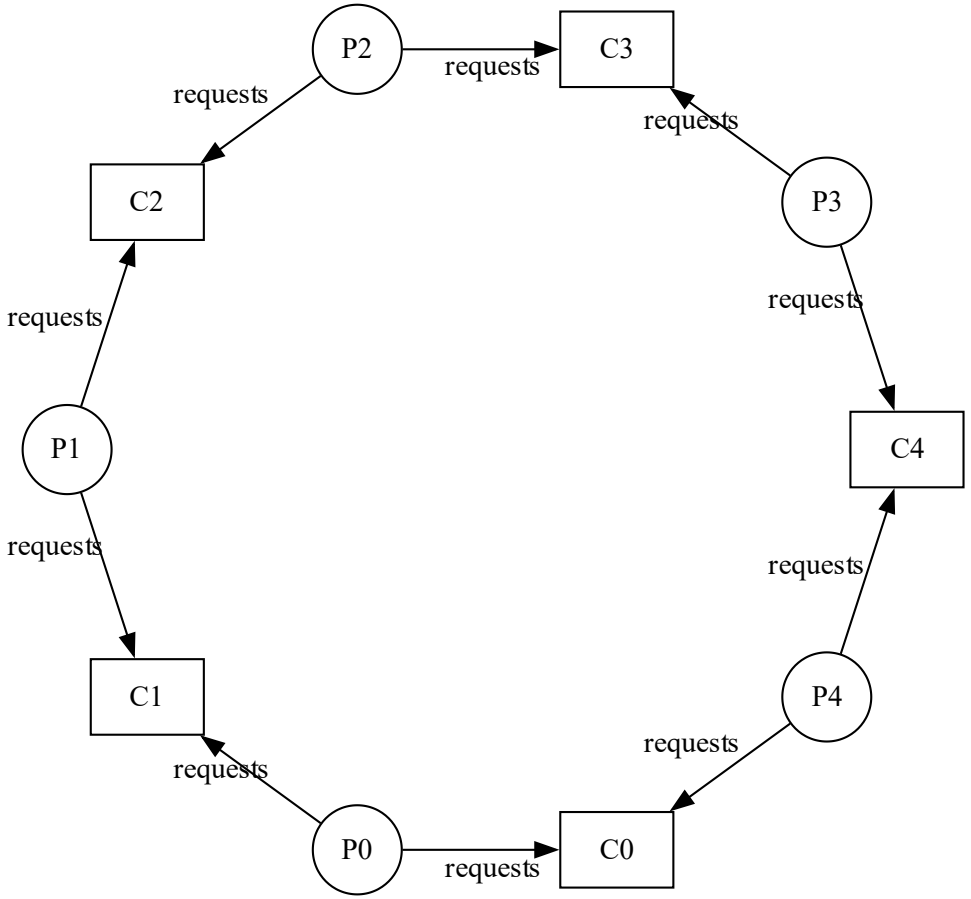
t	Phil 0	Phil 1	Phil 2	Phil 3	Phil 4
0	Pick C0				
1		Pick C1			
2			Pick C2		
3				Pick C3	
4					Pick C4
5	Pick C1 (locked!)				
6		Pick C2 (locked!)			
7			Pick C3 (locked)		



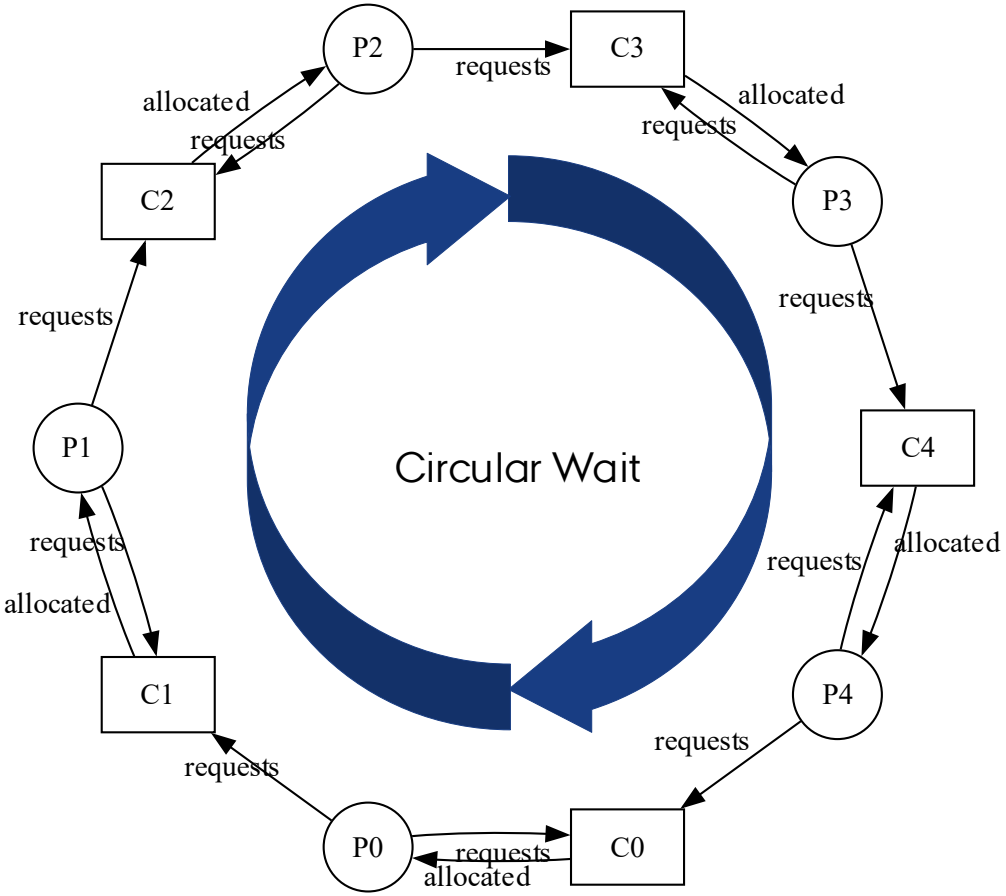
Deadlocked...

RESSORCE ALLOCATION GRAPH (RAG)

Initial State



In turn, each Philosopher picks a right fork



A SOLUTION?

Add additional requirements:

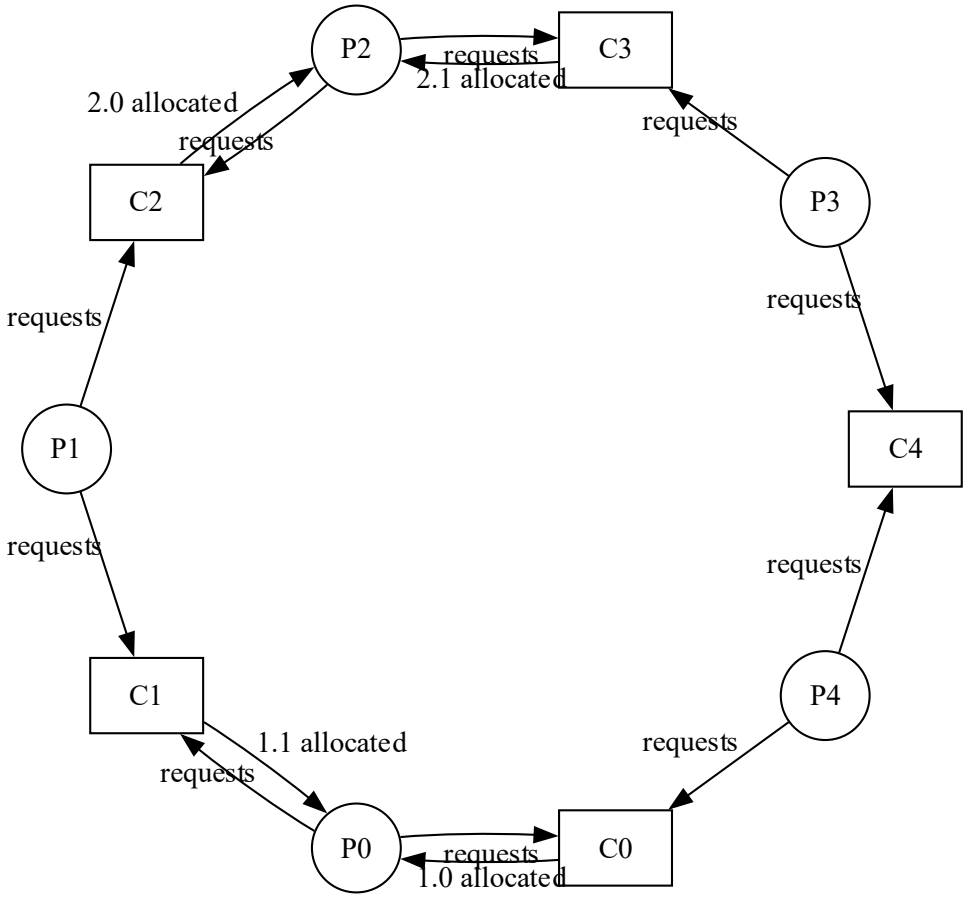
1) A philosopher must pick up the **lower-numbered chopstick first**

2) Then, If the higher numbered chopstick is not available, lower numbered chopstick must be released

No cycle → Solution*

*) When there is only one instance of each ressource (chopstick)

Will someone starve to death?



How can the additional requirements be implemented?

LESS NAIIVE IMPLEMENTATION

```
auto philosopher = [&](int phil)
{
  int right = phil;
  int left = (phil + 1) % 5;
  for (int i = 0; i < 100; i++) { // Eat 100 times
    while (true) {
      chopstick[right].lock();
      if (!chopstick[left].try_lock()) {// try_lock: peek at left chopstick
        chopstick[right].unlock(); // If L not available, release R
        continue; // Cont. to next while it (busy wait)
      }
      break; // Have both chopsticks, break out of while loop
    }

    std::cout << "Philosopher " << phil << " is eating" << std::endl;
    cnt[phil] = i;
    chopstick[right].unlock();
    chopstick[left].unlock();
  }
};
```

Will not deadlock, but will busy wait in `while(true)` loop

MUTEX IS NOT ALWAYS ENOUGH

A Mutex guarantees that a piece of code runs atomically, it is locked before critical code section and released afterwards

A Mutex however, cannot notify availability of a ressource to other threads. To check availability, we will have to poll (continously read) the resouce.

It would have been nice to sit and wait for another thread to notify the release of a chopstick and then continue, instead of continously reading (busy waiting).

So we need *condition variables*!

To use C++ notation, we first have to introduce some small classes that wrap a Mutex and make usage safer...

MUTEX WRAPPER CLASSES

`std::lock_guard` and `std::unique_lock`

(constructor)	constructs a lock_guard/unique_lock, optionally locking the given mutex
(destructor)	destructs the lock_guard/unique_lock object, unlocks the underlying mutex

cppreference.com

```
int main() {
std::mutex mtx;

auto thread_a = [&]() {
 const std::lock_guard<std::mutex> lock(mtx); // ~ mtx.lock()
 std::cout << "Hi " << "thread a";
}; // Scope ends: ~ mtx.unlock()

std::thread t(thread_a);
t.join();
}
```

They implement a **scoped lock**, the unique_lock has additional features

CONDITIONAL VARIABLES

A conditional variable (CV) is a synchronisation primitive to notify other threads that a shared ressource is available

A CV can:

Wait until it is notified

Notify one or many threads to wake-up

A CV is valuable because:

It avoids busy-waiting, and are thus more CPU efficient

It's safe, because it uses a Mutex to ensure exclusive access

C++ CONDITION VARIABLES

`std::condition_variable` can wait and notify:

Waiting:

```
void wait( std::unique_lock<std::mutex>& lock );
void wait( std::unique_lock<std::mutex>& lock, Predicate pred );
```

(Predicate is a true condition to check for, when waking up.
Used to manage spurious wake-ups)

```
while (!pred)
  wait(lock);
```

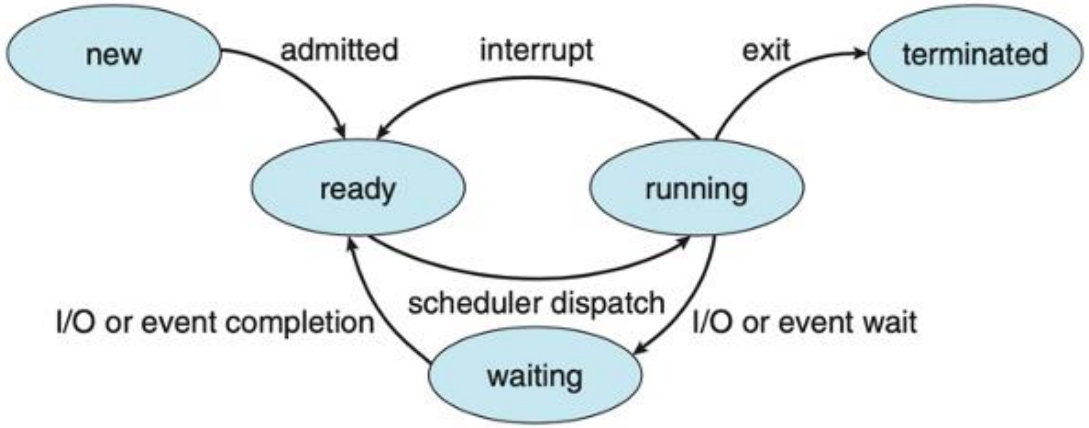
Notification:

`void notify_one();` (Notify ONE waiting thread)

`void notify_all();` (Notify ALL waiting threads)

CV INNER WORKINGS

Process State Diagram



`std::wait()`

1) Releases the mutex.

2) Puts the thread into the **Waiting** state.

3) The thread **sleeps**, waiting for a signal.

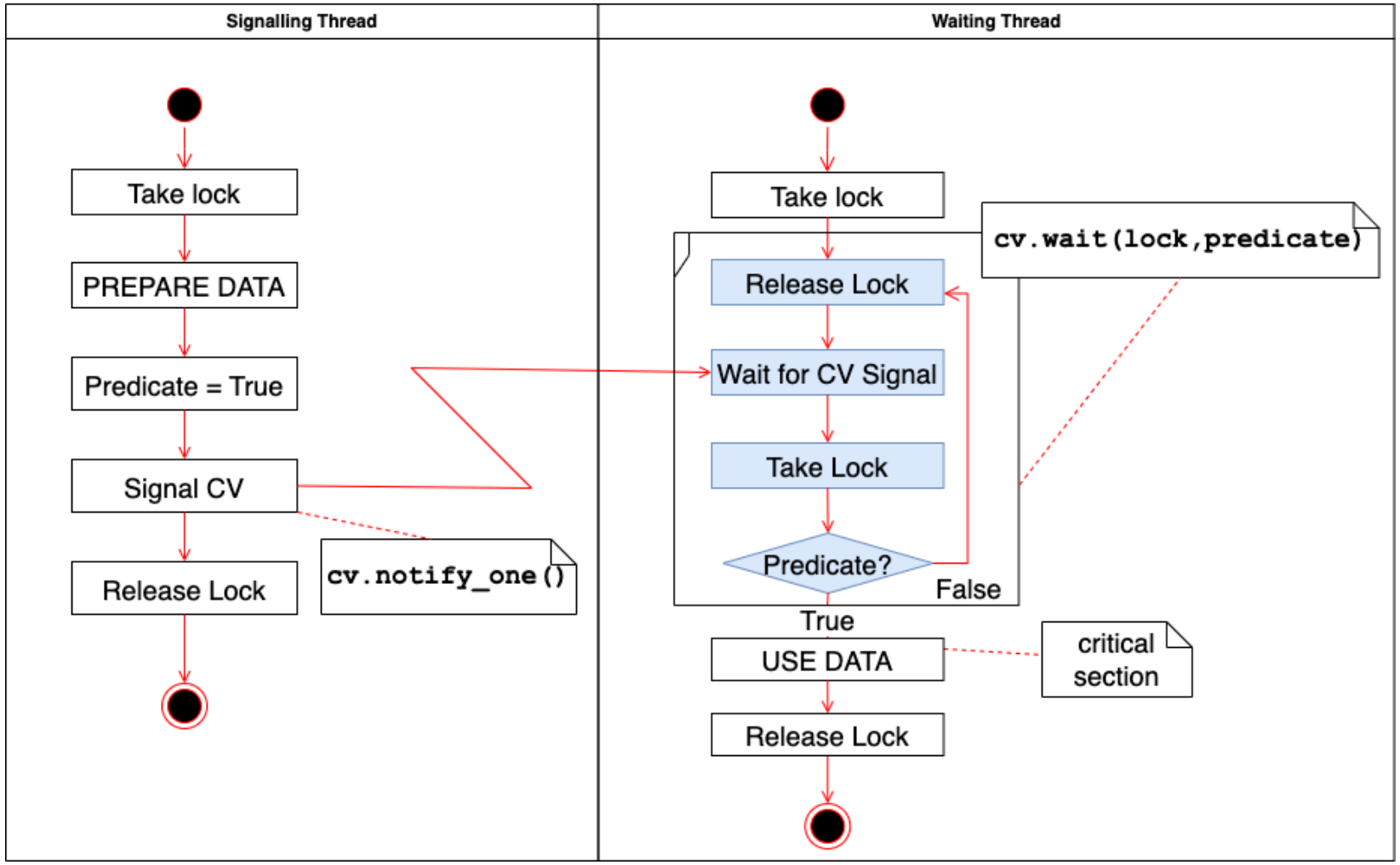
4) When woken-up, the thread becomes **Ready** and eventually **Running** again.

5) It automatically re-acquires the mutex before wait() returns (and potentially evaluates the predicate).

`std::notify_one() / std::notify_all()`

1) Wakes up **one/all waiting thread(s)** (if any) by signalling it to move from the **Waiting** state to the **Ready** state. The thread is now eligible to be scheduled by the OS.

CV SIGNALLING



CV EXAMPLE 1:2

```
condition_variable cv;
mutex cv_m; // This mutex is used for three
purposes:
// 1) to synchronize accesses to i
// 2) to synchronize accesses to cerr
// 3) for the condition variable cv
int i = 0;

void waits()
{
  unique_lock<mutex> lk(cv_m);
  cerr << "Waiting... \n";
  cv.wait(lk, []{ return i == 1; });
  cerr << "...finished waiting. i == 1\n";
}
```

Predicate:
Lambda return true if i ==1

Scoped lock

```
void signals()
{
  this_thread::sleep_for(chrono::seconds(1));
  {
    lock_guard<mutex> lk(cv_m);
    cerr << "Notifying...\n";
  }
  cv.notify_all();
  this_thread::sleep_for(chrono::seconds(1));
  {
    lock_guard<mutex> lk(cv_m);
    i = 1;
    cerr << "Notifying again...\n";
  }
  cv.notify_all();
}
```

Cppreference.com

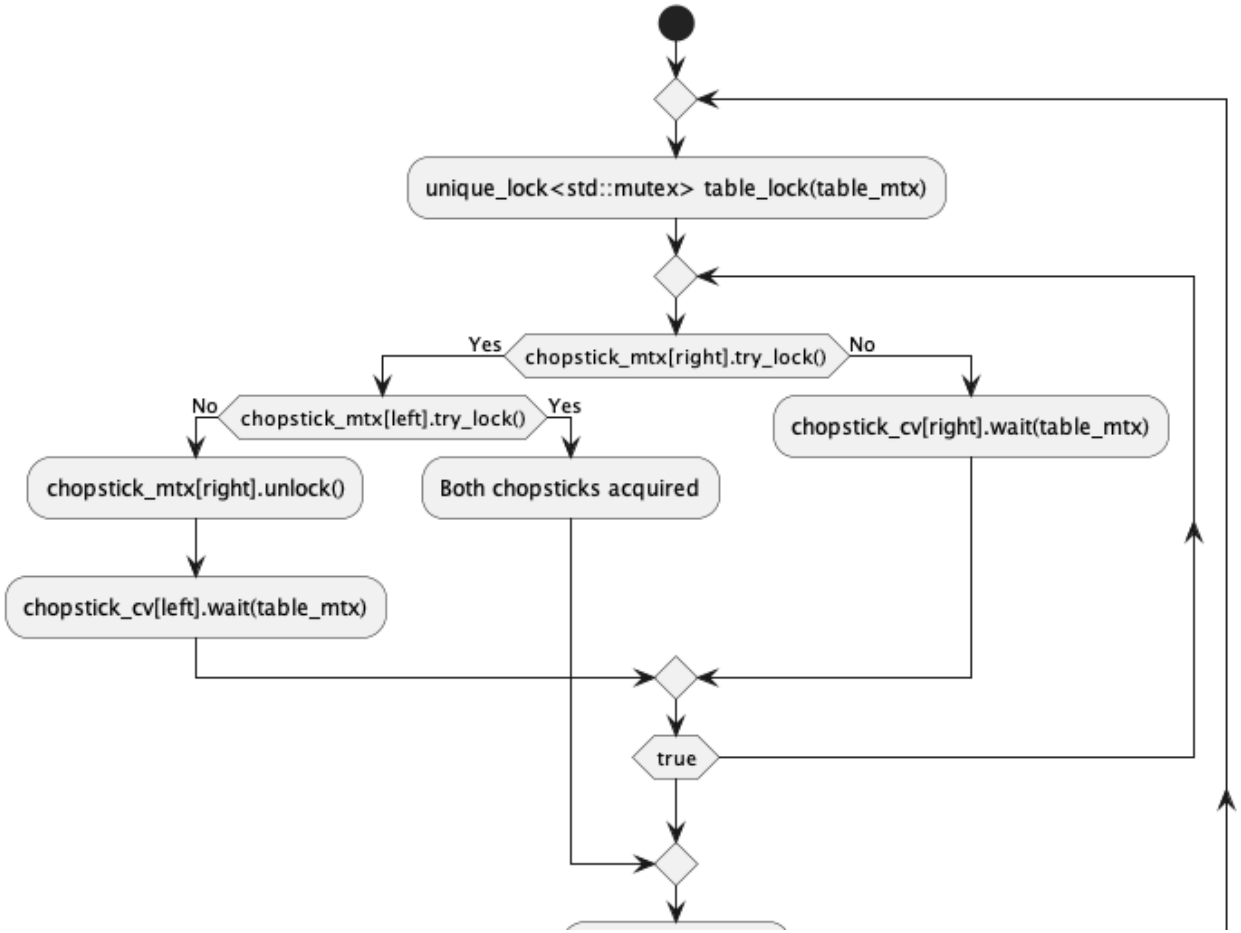
CV EXAMPLE 2:2

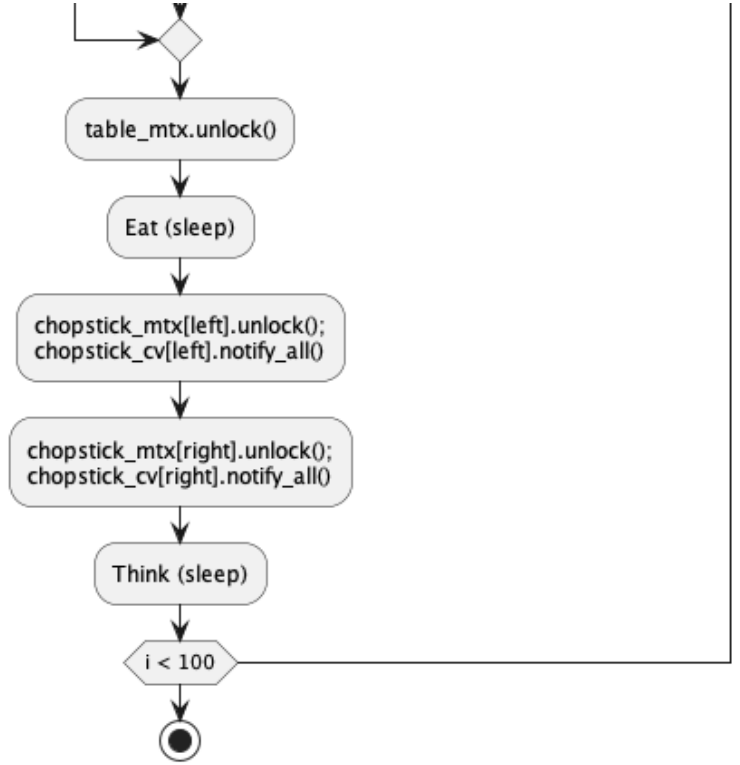
Waits	Signals	Mutex	CV State
`i=0` `unique_lock<mutex> lk(cv_m);`		lock	
`cv.wait(lk, []{ return i == 1; });`		unlock	waiting
	`{` `lock_guard<mutex> lk(cv_m);`	lock	
	`}`	unlock	
	`cv.notify_all();`		notify
`cv.wait(lk, []{ return i == 1; });` `(i==1? NO! -> Go back to wait!)`		lock -> unlock	wake-up: ready -> running -> waiting

	`{`
	`lock_guard<mutex> lk(cv_m);`
	`i = 1;`
		lock	

	`}`	unlock	
	`cv.notify_all();`		notify
`cv.wait(lk, []{ return i == 1; });` `(i==1? YES! -> Continue!)`		lock	wake-up: ready -> running
`}`		unlock	

DINING PHILS WITH CV





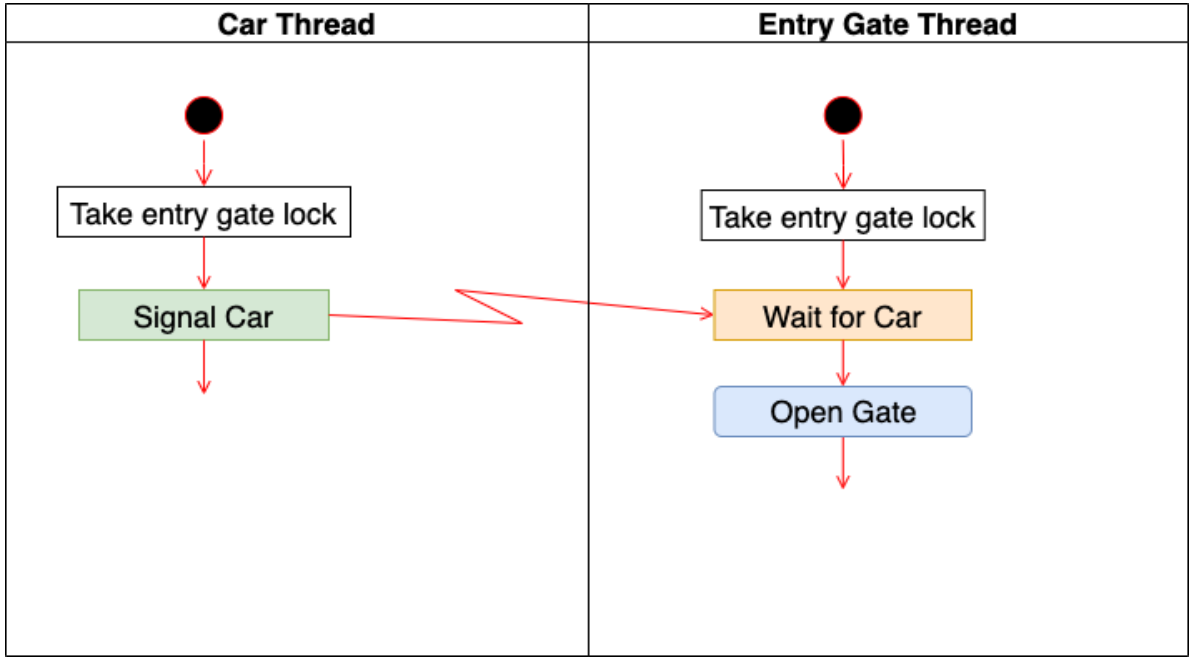
DINING PHILS WITH CV

```
auto philosopher = [&](int phil)
{
  int right = phil;
  int left = (phil + 1) % 5;
  // Eat 100 times!
  for (int i = 0; i < 100; i++) {
  // Lock table
  unique_lock<mutex> table_lock(table_mtx);
  while (true) {
    // If R mtx not availble, wait for R CV
    if (!chopstick_mtx[right].try_lock()) {
      chopstick_cv[right].wait(table_lock);
      continue; // Cont. to next while it.
    }

    if (!chopstick_mtx[left].try_lock()) {
    // L not available release R
      chopstick_mtx[right].unlock();
      chopstick_cv[left].wait(table_lock);
      continue; // Cont. to next while it.
    }
      break; // Both chopsticks acquired, break while
    }
    table_lock.unlock(); // Release table lock

    cout << "Philosopher " << phil << " is eating" <<
endl;
    cnt[phil] = i;

    chopstick_mtx[left].unlock();
    chopstick_cv[left].notify_one();

    chopstick_mtx[right].unlock();
    chopstick_cv[right].notify_one();

    cout << "Philosopher " << phil << " is
thinking\n";
    this_thread::sleep_for(chrono::microseconds(1));
// thinking
}
};
```

Asssuming namespace std

PARK-A-LOT-2000 (EXERCISE)

By Wfm at Dutch Wikipedia, CC BY-SA 3.0



Remember to use predicates

AARHUS
UNIVERSITY